Wiggle Sort

1. Non-recursive wiggle sort

This algorithm uses a for loop (iterative) without calling itself recursively. It reorders the array so that: $i_0 \leq i_1 \geq i_2 \leq i_3 \geq \ldots$ The user enters the array length (1-520) and values (0-5000), both validated with a while loop Memory is allocated dynamically with malloc and freed after use.

Pseudocode:

Pseudocode Line	Time Complexity
FUNCTION wiggle_sort(arraySize, array[])	O(1)
FOR i <- 0 TO arraySize - 2 DO	O(n)
IF (i even AND array[i] > array[i+1])	O(1)
OR (i odd AND array[i] < array[i+1]) THEN	O(1)
SWAP(array[i], array[i+1])	O(1)
END IF	-
END FOR	-
END FUNCTION	O(1)
BEGIN MAIN	O(1)
REPEAT	O(1)
READ length	O(1)
IF(length < 1 OR length > 520)THEN print error	O(1)
UNTIL length is valid (1 <= length <= 520)	O(1)
ALLOCATE array[length] via malloc	O(n)
count <- 0	O(1)
WHILE count < length DO	O(n)
READ value	O(1)
IF value < 0 OR value > 5000 THEN print error, CONTINUE	O(1)
array[count] <- value ; count <- count + 1	O(1)
END WHILE	-
CALL wiggle_sort(length, array)	O(n)
PRINT each element of array	O(n)

Complexity Analysis:

The FOR loop runs exactly n-1 times. Every operation inside (comparison + conditional swap) is O(1) The input validation loop is also O(n). The dominant term is O(n):

Total Time Complexity	O(n)
Space Complexity	O(1) _ only a single temp variable is used

2. Recursive wiggle sort:

This algorithm uses recursion — the function calls itself with array Index +1 until the base case (array Index >= array size -1)is reached. It applies the same wiggle condition but traverses the array via the call stack. The same dynamic input validation (length 1-520, values 0-5000) and malloc/free memory management are used. Each recursive call occupies a stack frame, so there is a risk of stack overflow for very large inputs in C

Pseudocode:

Pseudocode	Time Complexity
FUNCTION wiggle Sort(arrayIndex, arraySize, array[])	O(1)
IF arrayIndex >= arraySize - 1 THEN	O(1)
RETURN // base case	O(1)
END IF	O(1)
IF (arrayIndex even AND array[arrayIndex] > array[arrayIndex+1])	O(1)
OR (arrayIndex odd AND array[arrayIndex] < array[arrayIndex+1]) THEN	O(1)
SWAP(array[arrayIndex], array[arrayIndex+1])	O(1)
END IF	O(1)
CALL wiggSort(arrayIndex + 1, arraySize, array)	O(n)
END FUNCTION	-
BEGIN MAIN	O(1)

REPEAT	O(1)
READ length	O(1)
IF length < 1 OR length > 520 THEN print error	O(1)
UNTIL length is valid (1 <= length <= 520)	O(1)
ALLOCATE array[length] via malloc	O(n)
count <- 0	O(1)
WHILE count < length DO	O(n)
READ value	O(1)
IF value < 0 OR value > 5000 THEN print error, CONTINUE	O(1)
array[count] <- value ; count <- count + 1	O(1)
END WHILE	-
CALL wiggSort(0, length, array)	O(n)
PRINT each element of array	O(n)
FREE array	O(1)
END MAIN	O(1)

Complexity Analysis & Recurrence Relation:

Each call does O (1) work and makes one recursive call with index +1
This gives the recurrence:

*T(n) = T(n-1) + O (1)
*T(n) = T(n-2) + 2*O (1)
T(n) = T (1) + (n-1) O(1)
*T (1) = O(1) <- base case
*Therefore: T(n) = O(n)

Total Time Complexity	O(n)
Space Complexity	O(n) each of the (n-1) recursive calls occupies a stack frame. Risk of stack overflow in C for large n

Comparison

Comparison	Non-recursive	recursive
Type	Iterative (for loop)	Recursive (calls itself)
Time Complexity	O(n)	O(n)
Space Complexity	O (1) Only temp Var	O(n) Calls stack frames
Input Validation	While loop with bounds check	While loop with bounds check
Array Allocation	Dynamic malloc/free	Dynamic malloc/free
Max Length	1 to 520	1 to 520
Value Range	0 to 5000	0 to 5000
Stack Overflow	No risk	Risk for very large n for c
Best For	Large input, save production	Demon String Recursion

Conclusion: Both algorithms achieve O(n) time complexity and share the same dynamic input validation and malloc/free memory management. Algorithm 1 (non-recursive) is safer for large inputs because it uses O(1) space with no stack overflow risk. Algorithm 2 (Recursive) is more elegant and clearly demonstrates the recursive paradigm, but requires O(n) stack space and may cause a stack overflow for very large arrays in C.